

On the Ability of Graph Coloring Heuristics to Find Substructures in Social Networks

David Chalupa^{*}

Institute of Informatics and Software Engineering
Faculty of Informatics and Information Technologies
Slovak University of Technology in Bratislava
Ilkovičova 3, 842 16 Bratislava, Slovakia
chalupa.david@gmail.com

Abstract

Social network is a graph $G = [V, E]$, in which each vertex from V corresponds to a person and each edge from E describes, that two persons know each other. Social substructure of size m is defined as a clique K_m in social network G with m of its vertices. In other words, in a social substructure, everybody knows everybody. In this paper, we study the complementary graph $\bar{G}$, in which edges represent persons, who do not know each other. By solving the graph coloring problem with a proper heuristic over this graph, one can find social substructures without any additional information. We define this strategy as the uninformed approach to substructure discovery. Thus, the natural interpretation is assigned to the substructures by user of the method. Our experiments were performed using a well-known tabu search algorithm for the graph coloring problem, called TabuCol [6]. Experimental results are presented on several artificial social networks and one instance of real data.

Categories and Subject Descriptors

G.2.3 [Discrete Mathematics]: Applications; I.2.6 [Learning]: Knowledge acquisition; I.2.8 [Problem Solving, Control Methods, and Search]: Heuristic methods

Keywords

Graph Coloring, Social Networks, Substructures, Tabu Search

^{*}Master degree study programme in field Software Engineering. Supervisor: Professor Jiří Pospíchal, Institute of Applied Informatics, Faculty of Informatics and Information Technologies, STU in Bratislava.

Work described in this paper was presented at the 7th Student Research Conference in Informatics and Information Technologies IIT.SRC 2011 and awarded by Dean's Award.

© Copyright 2011. All rights reserved. Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from STU Press, Vazovova 5, 811 07 Bratislava, Slovakia.

Chalupa, D. On the Ability of Graph Coloring Heuristics to Find Substructures in Social Networks. Information Sciences and Technologies Bulletin of the ACM Slovakia, Special Section on Student Research in Informatics and Information Technologies, Vol. 3, No. 2 (2011) 51-54

1. Introduction

Social network is a graph representing relationships between people. Such graph is a couple $G = [V, E]$, where vertex set V represents people and edge set E contains couples of vertices, representing that people know each other.

Social substructure is a clique K_m with m vertices of social network. Thus, in a social substructure, everybody knows everybody. At this point, we should mention that deciding, whether there is a clique of a particular size in a graph, is an NP-complete problem. Thus, this problem generally needs exponential time to be optimally solved, it is hard to approximate and practically must be solved by sophisticated heuristics [8].

Suppose that no metadata is available and substructures have to be searched for only according to the topology of the graph. We will refer to this case as to the *uninformed approach* to substructure discovery. In this approach, a heuristic algorithm is first used to search for social substructures and the result is interpreted by user of the method. The most interesting motivation to use the uninformed approach is that regarding only the topology, one might be able to discover substructures that cannot be intuitively found by informed approach. However, the uninformed approach is apparently reducible to a computationally hard problem. We have chosen to transform it to the *graph coloring problem*.

2. Related work

For the graph coloring problem, there are many heuristics that can be used for the problem we study. The most straightforward approach are the greedy heuristics, building the coloring iteratively. The most well-known greedy algorithm for the graph coloring is the *Brélaž heuristic* [2]. In this paper, we use a *local search algorithm*. Local search generally begins with a random coloring and improves it with a sequence of elementary moves [5]. In a so-called *tabu search*, inverse move is forbidden for several iterations. In our approach, we use the most popular tabu search algorithm for the graph coloring, called *TabuCol* [6].

An alternative approach is represented by the *clustering algorithms*. In clustering, groups of instances in a sample are detected using similarity measures. When we want to use clustering, we have to either represent the instances by vectors, or directly define distances between them. On the other hand, the graph-based approach, we deal with

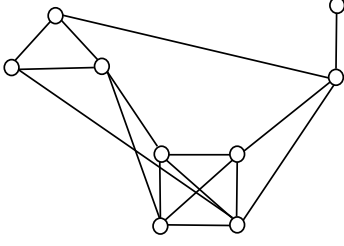


Figure 1: An example of a typical social network topology.

in this paper, uses only a binary information, represented by edges. The most well-known clustering algorithms are the k-means algorithm and the hierarchical clustering [9].

3. Topology of social networks

Social networks are topologically specific graphs, consisting of cliques, connected by more sparse areas of the graphs. Regarding just the topology, we will assume that people, who know each other, have an edge between their vertices in the social network. Detection of missing edges is another interesting problem, however, this paper is not aimed to deal with it. A small example of a typical social network topology is depicted in Figure 1.

In order to simulate behavior of graph coloring heuristics on topologies we discuss, we have designed a simple model, which can be used to generate artificial social networks. Suppose we have $|V|$ vertices and c substructures. We define p_c as a probability that a vertex is in a substructure. To provide simplicity, we assume that p_c is equal for each substructure. Using these parameters, we can generate a binary matrix B such that its element $b_{v,k}$ denotes if vertex v is in substructure k . Then, an edge is put between two vertices if they are in at least one common substructure. Otherwise, an edge is put between them only with some small probability p_m , in our case 0.01. We can describe this model as $[|V|, c, p_c, p_m]$.

Practically, this model generates quite typical topologies, in compare to real data we also used in one experiment. In our experiments we use $cp_c = 1$ to ensure that a vertex will be in a single substructure in average case. However, this model has one major drawback. For example, in model $[20, 5, 0.2, 0.01]$, probability that a vertex will not be in any substructure is $(0.8)^5 \approx 0.32$, thus producing many isolated vertices. This drawback is overcome simply by a postprocessing step, when each isolated vertex is inserted into a randomly chosen substructure.

4. The uninformed approach to substructure discovery

First, we formally define the *graph coloring problem*. Let $G = [V, E]$ be an undirected graph and let c be a number of colors. The objective of the graph coloring problem is to find a partitioning of the vertex set V into partitions $V_1, V_2, \dots, V_c$ such that the partitions cover the whole vertex set and the number of adjacent vertices in the same partitions is minimal:

$$J = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \text{confl}(i, j) \rightarrow \min, \text{ where} \quad (1)$$

$$\text{confl}(i, j) = \begin{cases} 1 & \exists k : v_i \in V_k \wedge v_j \in V_k \wedge [v_i, v_j] \in E \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

In other words, the objective of the graph coloring problem is to find internally independent partitions, where no couple of vertices is connected with an edge. When we fix the number of colors c , *fitness function* can be defined simply as $F = |E| - J$. When two connected vertices are equally colored, we will refer to this situation as to a *collision*.

4.1 Basic steps of the method

The general idea of our approach to substructure discovery is based on construction of a complementary graph and coloring it with as few colors as possible. Thus, our method tackles the problem indirectly. Instead of searching for cliques, it "breaks" the complementary graph into independent sets. It consists of the following four steps.

1. We choose a group of candidates and extract information about contacts of each candidate. We construct topology of the network as a graph G .
2. We create a complementary graph $\bar{G}$ and store it in a standard DIMACS format [7]. In the complementary graph, an edge means that people do not know each other. This is needed because we do not want to minimize but maximize dependence in the resulting partitions.
3. We solve the graph coloring problem over the complementary graph $\bar{G}$ with c colors, where c is a parameter. By repeated resetting of this parameter, we minimize its value. Thus, we minimize the number of substructures.
4. The coloring we find represents a partitioning of the vertex set into c subsets. Each subset contains vertices of the same color. We assign people to each of the vertices. Natural interpretation of the partitioning is done by user of the method.

One can easily see that steps 1, 2 and 4 are quite trivial. However, in step 3, various graph coloring heuristics can be chosen. In the next section, we focus on the algorithm we have chosen.

4.2 Graph coloring heuristic

Heuristics, which can be used, vary from simple greedy algorithms [2] to complex hybrid evolutionary algorithms [4] and methods simulating insect behavior [3]. In our work, we have chosen the most well-known tabu search algorithm for the graph coloring problem, very popular due to its balanced simplicity and performance, also known as TabuCol [6].

TabuCol works as follows. First, an entirely random coloring is generated. Then, an iterative procedure is performed. In each iteration, we have an actual coloring S , on which a mutation is going to be performed. A *conflicting vertex* is a vertex, which is involved in at least one collision. A *neighborhood* $N(S)$ is then defined as a set of all colorings obtained from S by recoloring every conflicting vertex with every other possible color. Each of the

Table 1: Computational results of TabuCol on 3 different artificial networks

G	$[V , c, p_c, p_m]$	c	succ.	st. $\times 10^6$	CPU
<i>artif_100_20</i>	[100, 20, 0.05, 0.01]	20	10/10	< 1	< 1 s
<i>artif_500_100</i>	[500, 100, 0.01, 0.01]	100	6/10	26	3 s
		99	2/10	44	7 s
<i>artif_2000_400</i>	[2000, 400, 0.0025, 0.01]	400	4/10	1072	51 s
		399	1/10	1201	64 s

colorings in $N(S)$ is evaluated by the fitness function and the one with the highest fitness is chosen as a new actual coloring S^* . This process is iteratively repeated until a maximal number of iterations is reached or an optimal coloring is found. If a tabu move leads to a coloring that has a higher fitness than any coloring found so far, tabu search also accepts it [6].

When we perform these moves, it is quite easy to get trapped, cycling in an uninteresting region of the state space. Suppose that vertex v had a color c_1 and the search algorithm recolored it with c_2 . Thus, the move is defined by the couple $[v, c_2]$. However, the inverse move $[v, c_1]$ would lead the search back to the previous state. Thus, the tabu search forbids this move for a number of iterations defined by the tabu tenure [6].

The tabu tenure can be defined in various ways. In current graph coloring literature, one can find mostly the dynamic tabu tenure, using the number of conflicting vertices as a metric for dynamic calculation [4]. Recent developments also introduced the reactive tabu tenure, using fluctuation of the fitness function to determine, when to reactively increment the tabu tenure to prevent cycling [1]. However, for our purpose, we define the tabu tenure only simply as a random integer ranging from 0 to 20.

5. Experimental results

In this section, we present experimental results of our method on 3 artificial social networks and one instance of real data. The first part is aimed to present the computational results of this approach. The second part presents the natural interpretation of the obtained colorings.

5.1 Results on artificial instances

With regard to the fact that this paper deals with entirely new instances, we wanted to keep the experimental conditions as simple as possible. Using our model, we generated 3 artificial social networks with 100, 500 and 2000 vertices. We tried to solve them with a very simple version of TabuCol algorithm [6] with randomly generated tabu tenure ranging from 0 to 20. We stopped the algorithm either when a proper coloring was found or 10^5 iterations were performed. An example of a result on the network with 100 vertices and 20 substructures, grouped to circles, is depicted in Figure 2.

Table 1 summarizes results we obtained. First two columns contain the name of the graph and parameters of the model, which generated the instance. The next columns contain the number of substructures we tried to find (in other words, the number of colors we used), the success rate (number of successful runs out of 10), average number of colorings that were evaluated in millions, average number of iterations that were performed in thousands and average CPU time the algorithm used.

On the network with 100 vertices, the algorithm found 20 substructures very quickly. On the network with 500 vertices, this simple algorithm still performed quite reliably, when searching for 100 substructures that were guaranteed to exist. On the other hand, larger instances seem to generate very large plateaux in the state space of the instance. Thus, a simple algorithm like this begins to suffer from lower success rate. It is also worth mentioning that there exists even a solution with 98 substructures in this graph, due to the probabilistic nature of the model it is was generated with (some of the substructures can be simply merged). However, we obtained this coloring only with more sophisticated algorithms.

Even the largest graph was still solved by the simple algorithm, the expected 400 substructures were found on this very large instance. The algorithm even managed to find 399 structures on one run. On one hand, this shows that this methodology is able to find encouraging results even on graphs with 2000 vertices, which are considered to be huge in measures of current graph coloring algorithms. On the other hand, it still makes sense to deal with more sophisticated heuristics, due to the success rate issues.

5.2 Result on a real instance

Finally, Figure 3 presents result obtained on an instance, representing real data obtained from social network Facebook¹. This graph contains 52 vertices and shows the substructures grouped to circles. All these substructures do have their natural interpretation and are relevant according to the data, from which the graph was constructed.

First, in the lower right part of the graph, one can see three substructures with very dense connection. These substructures are strongly connected to each other and practically have the same natural interpretation. Their separation was caused by the fact that some people, who know each other, do not have themselves in their contact lists. This indeed can happen more probably in larger groups of people. However, the figure shows that such phenomenon can be detected visually during the interpretation. Moreover, such a phenomenon can point to smaller substructures within larger, closer groups of people.

The left part of the graph consists of several smaller, more sparsely connected substructures. These substructures represent more loosely connected groups of people. However, they still have very similar natural interpretation. Typically, this situation can be found in very large organizations with smaller departments. In this case, we might say that these substructures are "distributed".

¹<http://www.facebook.com>

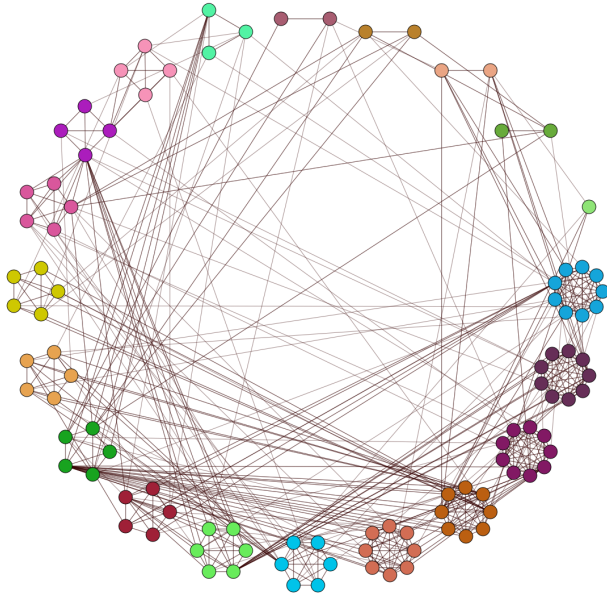


Figure 2: Result obtained on artificial network *artif_100_20* with 100 vertices and 20 substructures.

The last part of the graph shows more isolated substructures. In this case, the substructures have quite different and usually very clear natural interpretations. In fact, these substructures are the easiest to find in the topology, simply because they are usually small and do not depend on other substructures. This is also why also the natural interpretation of these substructures seems to be the easiest.

6. Conclusion

First of all, the aim of this paper was to introduce a methodology, how to find information in graphs generated by a modern real world application. We have shown that such topologies can be reliably simulated by a simple probabilistic model. Moreover, we have experimentally tested a simple graph coloring heuristic on such instances, to determine their nature. Although these graphs can be solved even with a simple heuristic, we have shown that there is still an open problem of success rate that can be tackled by more sophisticated algorithms. Regarding graphs with more than 2000 vertices, there still is a space for further improvement. Particularly, it might be interesting to use directly the social network and solve the complementary problem, instead of graph coloring. This approach might decrease memory demands, however, it would not improve computational results.

Moreover, we have shown that our methodology produces results of high relevance for real data. This methodology was able to classify people, using only topology of the social network. This uninformed approach might be very interesting when one wants to find substructures that cannot be identified intuitively using metadata about people in the social network.

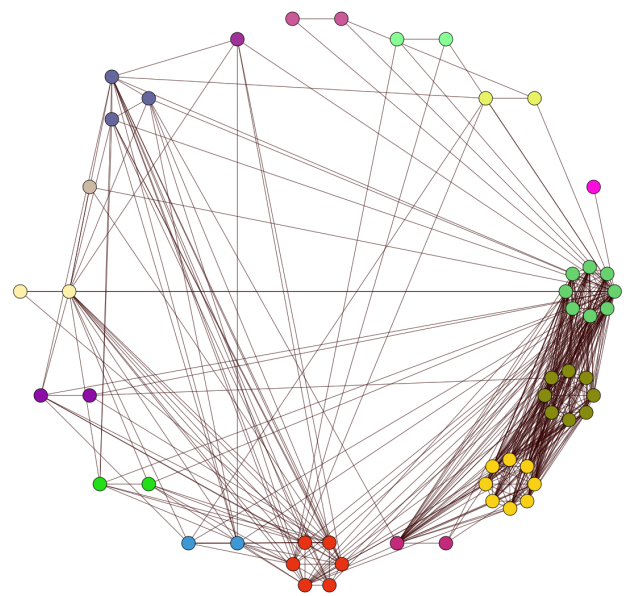


Figure 3: Result obtained on a sample of real data obtained from social network Facebook.

Finally, social networks and complementary graphs can be very interesting also as new graph coloring instances. They might be used as an extension of current DIMACS benchmark graphs, used in the graph coloring literature [7].

Acknowledgement. This contribution was supported by Grant Agency VEGA SR under the grant 1/0141/10.

References

- [1] I. Blöchliger and N. Zufferey. A graph coloring heuristic using partial solutions and a reactive tabu scheme. *Computers and Operations Research*, 35(3):960–975, 2008.
- [2] D. Brélaz. New methods to color vertices of a graph. *Communications of the ACM*, 22:251–256, 1979.
- [3] D. Chalupa. An optimization Strategy Based on Social Insect Behavior. In *SOFSEM 2011 Student Research Forum*, pages 35–50. Okat, 2011.
- [4] P. Galinier and J. K. Hao. Hybrid Evolutionary Algorithms for Graph Coloring. *Journal of Combinatorial Optimization*, 3:379–397, 1999.
- [5] P. Galinier and A. Hertz. A survey of local search methods for graph coloring. *Computers and Operations Research*, 33(9):2547–2562, 2006.
- [6] A. Hertz and D. de Werra. Using tabu search techniques for graph coloring. *Computing*, 39(4):345–351, 1987.
- [7] D. S. Johnson and M. Trick. *Cliques, Coloring, and Satisfiability: Second DIMACS Implementation Challenge*. American Mathematical Society, 1996.
- [8] R. M. Karp. Reducibility among combinatorial problems. *Complexity of Computer Computations*, pages 85–103, 1972.
- [9] R. Xu and D. Wunsch. Survey of Clustering Algorithms. *IEEE Transactions on Neural Networks*, 16:645–678, 2005.